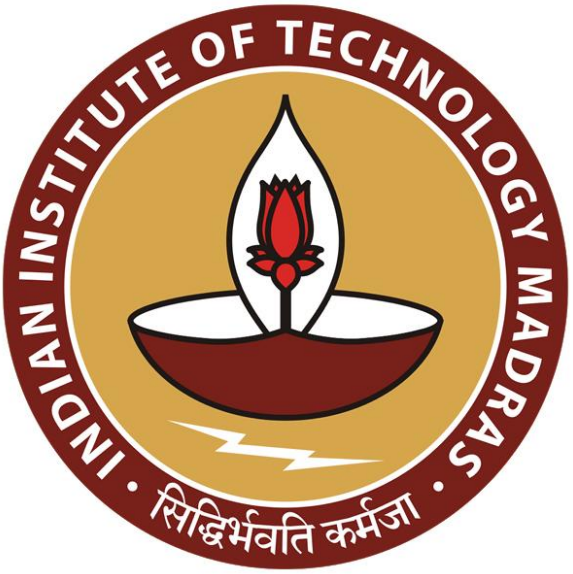


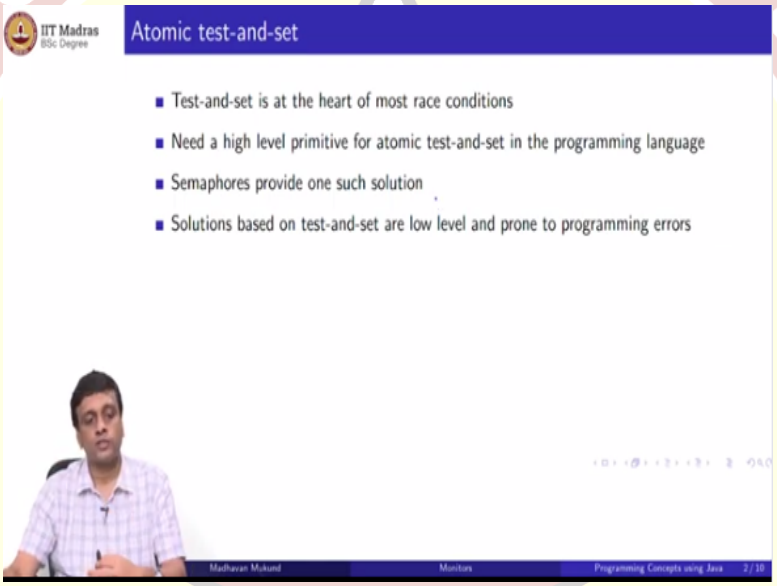
IIT Madras

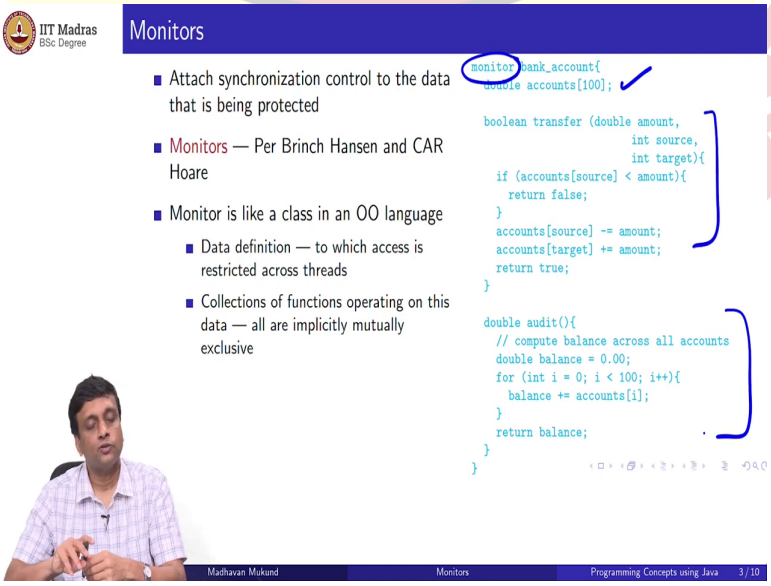
ONLINE DEGREE

Programming Concepts using Java
Professor. Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology, Madras
Monitors

So, we are looking at programming language support for synchronization. So, we moved from this very elementary protocols using arbitrary shared variables like Peterson's algorithm or the bakery algorithm, we move to semaphores, which gave us this test and set operation which was atomic.

(Refer Slide Time: 00:30)





So test and set, we said is really the key. So, once we have something which gives us test and set, then we can, in principle program these mutually exclusive concurrent critical sections. But there is a lot of loose ends in terms of requirements of what the programmer will oblige to do and discipline and so on. So, what we can do next is to actually move to something much more structured, and that is what is called a monitor.

So, in a monitor, we combine, and this is really what we want to do, we want to say here is data that should be protected from arbitrary axis. And here are the functions which do that arbitrary axis. And I want to make sure that when you call these functions, these are synchronized with respect to the data. So, this is now ideal for what we would call an abstract data type, we want to say that this, these functions, and this data must be put together in a package, which guarantees some kind of uniform update, some kind of integrity of the data.

So this is the same way when we say that you take a stack, and you take its implementation main array, a linked list, and you take the push pop, and you do not allow the push pop to be segregated from this so that you have access to the array of the linked list independently, and then you are able to manipulate it, you put them together, and you say, you can only access this data through these functions, and therefore you maintain integrity.

Now we are adding another constraint, you can only access this data through these functions. And these functions cannot interfere with each other. So, that is what a monitor is. So, monitors were introduced by two people Per Brinch Hansen, who is again, an operating systems person, and Tony Hoare, who is probably more famous for having invented Quicksort. But again, a person who was deeply interested in concurrent programming for most of his life and systems, software, and so on.

So, what is a monitor? Well, a monitor is basically as we can see, as we described, is basically an abstract data type, or in our context, since we are working in Java, it is like a class. But, there is some extra connotations associated with the class, it is not so much about public, private and capsulation, all that, of course, all that is there, but it is really about concurrent access.

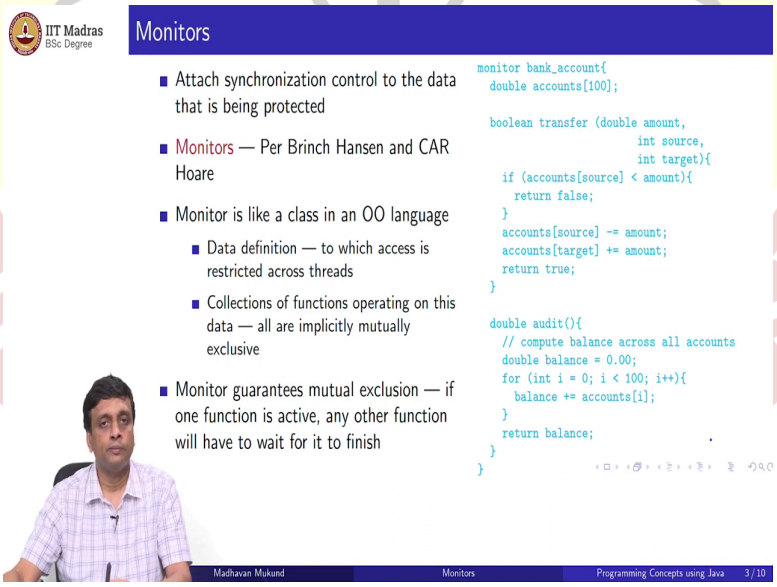
So, when we have a class, what do we normally have, we have a data description, the instance variables, and we have the methods. So, remember our earlier account space thing, we had this bank account, which is an array of 100 doubles. And we wanted to have this

transfer and audit functions which were available on this. So, earlier, we had two separate things, so we had an audit class.

And I mean, the functions could operate on this audit class. So, it is more or less the same syntax, except that instead of class, we just for the moment, mean, this is not really Java code. This is more kind of hypothetical pseudo code, which looks a little bit like Java, but instead of declaring to be a class, we are declaring it can be a new type of thing called a monitor. So, like a normal class, it has the instance variables, and it has the functions that we wrote before nothing has changed.

The only thing that has changed is that I am instead of calling it a class, I am calling it a monitor. And the reason that this is important is that once I put these functions together inside a monitor, what it tells the programming language support is that these functions have to be executed atomically with respect to this, so these functions cannot be executed concurrently on this object. So, if I start a transfer on on a particular bank account, I cannot simultaneously start an audit or if an audit is running, then the transfer has to wait, so this is a built in feature of monitors.

(Refer Slide Time: 04:11)

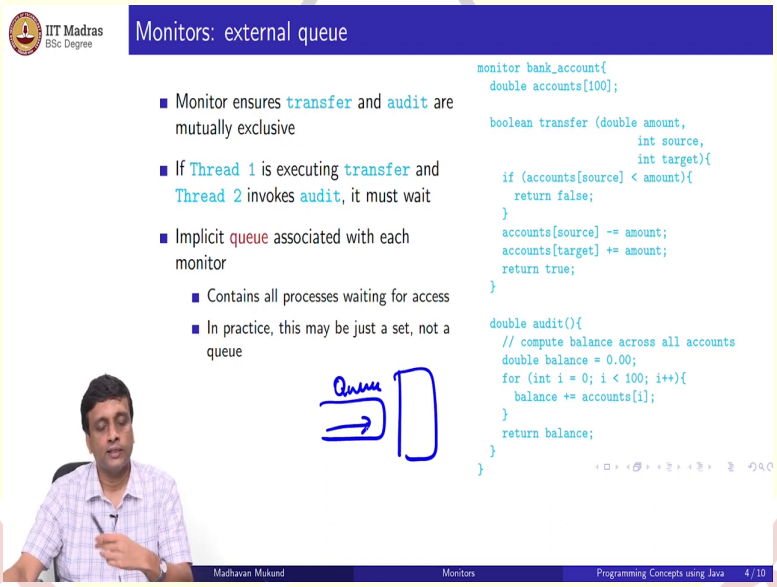


So, monitors therefore by it is, it is just a language design, there is no nothing to be proved. It is the meaning of a monitor is that the functions associated with that monitor object will be guaranteed to be executed in a mutually exclusive fashion, no to objects, no two functions on

the same object will happen simultaneously. So, if one function is active, and you call another function, then it will basically get stuck.

You will have to wait for the first function to terminate. So, what we will see is what it means for functions to terminate and wake each other up and so on. But at the high level, once you put these functions together, the programmer does not have to think about it anymore. So, now you are guaranteed that for a particular bank account object created from this monitor, think of monitor as a kind of plus with some extra thing, so it is just a class. So, I can create an object of this type. And it will come with these two functions with the extra promise that if I run them in a thread, these two, these two methods will not ever be executed simultaneously by two different threads.

(Refer Slide Time: 05:15)



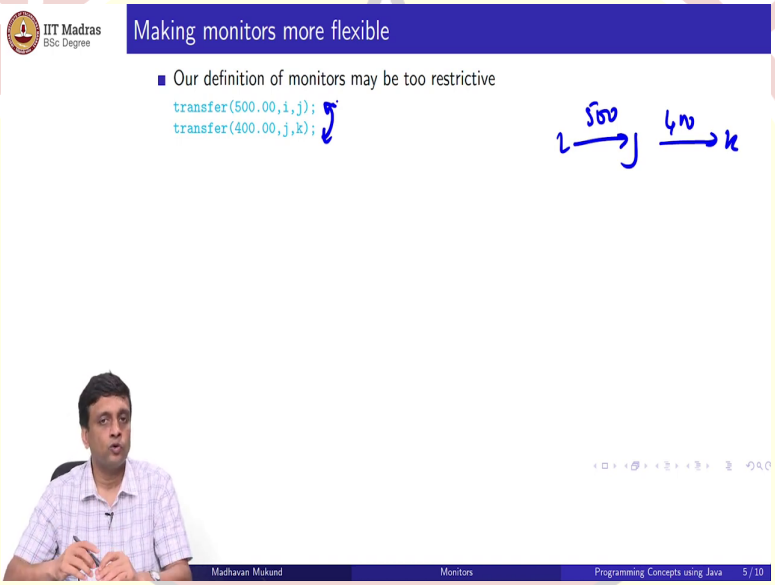
So, what in this example the monitor does is it ensures that this transfer is independent of this audit. So, if one is executing transfer, the other one executes audit, it must wait. So, what this means it has to wait somewhere. So, there is associated with every object now some kind of queue of, of requests.

So, threads waiting to execute something on this object, which they have not been allowed to because somebody is already there inside. So, imagine you want to meet your doctor, you are sitting in the waiting room because somebody is already being examined by the doctor. So, only when that person comes out, can the next person go in.

And so if it is a regular queue, then it will happen in the order in which you came. But it is not guaranteed that this is so we call it a queue. But it need not be a queue. They could be different ways in which this is organized. But essentially there is a waiting room, it may be a waiting room is a better term for it. But traditionally, it is called a queue.

So, you have an external queue or an external waiting room, where all the threads which are trying to get access to this object to the function to the object is waiting until the currently executing function releases the object, and then you can proceed, so this is how a monitor works. So, there is this external queue or external waiting room where everything waits.

(Refer Slide Time: 06:36)



Now let us look at our old example again. So, suppose we have some bank account with 100 bank, bank accounts, and then we execute two transfers, concurrently. Now we know that these two transfers cannot interfere with each other. So, the outcome of this transfer is going to be fixed. So, what we are doing here is we are transferring 500 from i to j. So, 500 from i to j. And then we are transferring 400, from j to k.

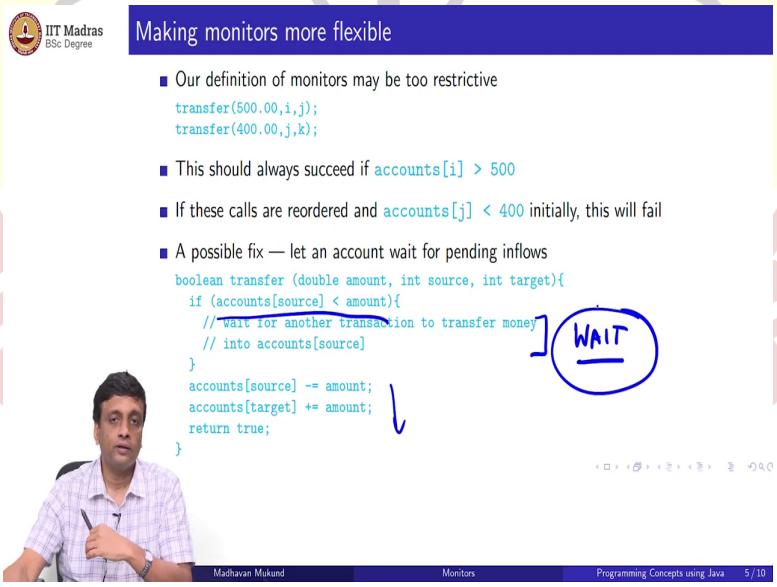
So, since j is getting 500, it does not matter really what j had before even if j did not have 400 before. Certainly after it gets 500, it has enough to make the transfer. So, logically, these two transfer should go through. If I execute the transfer from i to j followed by j to k, we should go through. But now the problem is, of course, these might be in two different threads. So, these might happen out of order. So, I might try to transfer 400 from j to k, before I transfer the 500 from i to j. And you can imagine that this might happen in a real bank.

See when the person comes to work. Nowadays, of course, things happen on NEFT and all that. So, you do the electronic banking, and things happen overnight, and so on. But usually in a traditional bank, if you are writing cheques, for example, then on a given day, there will be a number of cheques which are awaiting processing for that day. Some of these cheques will be outgoing from your account, some of these cheques will be incoming into your account.

So, say the salary day, there is a cheque which is from your employer, which is paying into your account, and then you have also written a cheque to pay your rent. Now, if the person tries to pay the rent, before the salary is credited, maybe your account will then go negative. But the bank has to resolve this. So, the bank will usually try to make sure that all the deposits are made before the payments are made.

But what if the deposits are from one to another. Your deposit may be coming from another person's payment. So, there that payment will have to wait. And so it is not very easy to reconcile this. So, there may be no no sensible way to make sure that you find an order in which you can actually execute all these things without blocking.

(Refer Slide Time: 08:44)



So, since this should always succeed, how do we deal with the situation where it fails? And allow it to retry in some sense. So, we want to say that okay now, this 400 transfer is not working, because the 500 I know is going to come maybe I do not know, but I am hoping that something will come to release it. So, let me put this aside and come back and try it later. So,

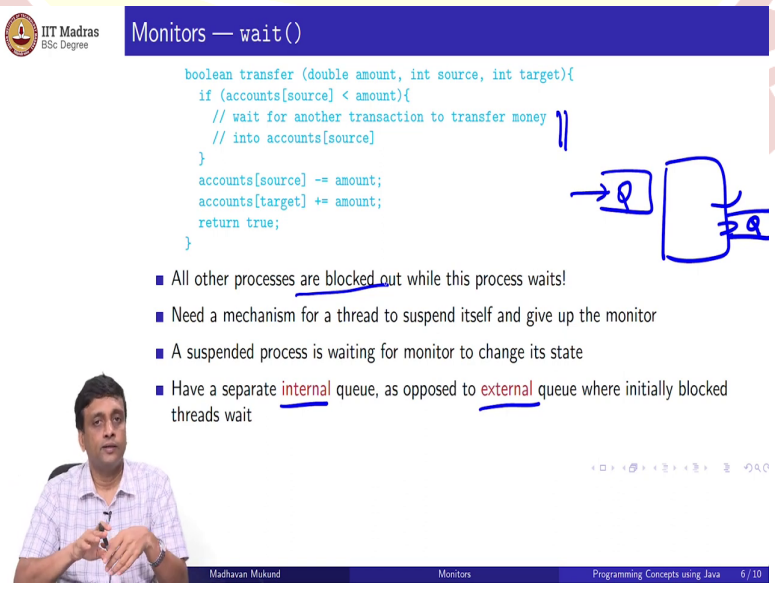
if we do this 400 transfer before the 500 transfer and j does not have enough balance, we have a problem.

So, what we would really like is a kind of waiting mechanism. So, I tried to cheque whether the source account has enough to transfer and in the earlier situation the simple explanation that I had before the simple implementation, if this was not the case, we just return false we say this transfer not possible. Now, we are saying that maybe that is a bit too harsh, maybe it is possible provided something comes to balance out this transfer. So, I want to wait for it. And once that happens, I will go ahead and do the transfer.

So the question that we are going to talk about now is how to introduce in this monitor mechanism. So, the monitor mechanism otherwise will say you cannot just wait here that is another problem. I mean, if I am just hanging around here, notice that there is only one accounts array. Anybody who is to put money into those accounts array needs access to this transfer function. But I am sitting in this transfer function preventing everybody from access.

So, I want something to happen to make this transfer possible. But I cannot make the transfer happen until I get out. So, I need to get out to allow somebody else to transfer money into this account so that my 400 transfer can go through. So, I need a way of exiting the monitor without starting from scratch, I need a way of saying that I am waiting and please let me know if something else happens. So, I need a graceful way to sit outside and wait for something to happen without terminating this function.

(Refer Slide Time: 10:48)



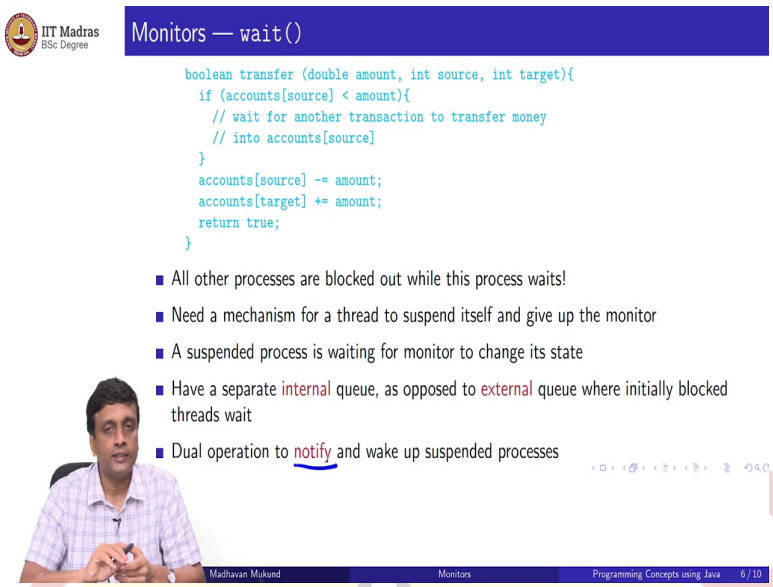
So, all other processes are blocked out while I wait. So, therefore, I need to suspend and release the monitor for somebody else to proceed. So, I need a kind of a different waiting room. So, I am waiting for the process to change. So, I cannot be waiting outside. Because I am waiting outside, if I have never had access to the process, remember, I had this object. And I had an outside queue, where I am waiting for access.

Now I came inside here and I wanted to wait somewhere else. So, there must be an inside queue. So, these are people who had, these are processes or threads, which started doing something and then they realized that there is something not quite correct about the state of the object, so I cannot finish my job. But it is possible that some other objects will come and some other threads will come and fix the problem for me. So, let me wait till then.

So, we have an internal queue as opposed to the external queue. So, the external queue are threads, which have never started it. So, they came to try to do something and there was already some other thread executing inside this object. So, they were waiting. Internal queue is threads, which came in did something halfway like this, they came to a situation where they got stuck. And they felt, this is not a deadly situation, maybe I can get out of the situation if only someone will cooperate with me. So, let me wait for somebody to come and rescue me.

So, then they have to still step aside because if they do not step aside, then nobody else can make progress, everything else is blocked. So, because only one thread can execute inside a monitor at one time, so if I am sitting there, I have to wait, I have to have a way of suspending myself and taking myself out of the picture to let the other thread through.

(Refer Slide Time: 12:30)



So of course, now the symmetric thing must happen to wake up. So, remember earlier, we said even may look at semaphores that when we release a semaphore, you wake up the people who are waiting. So, there is an implicit idea here that when one thread exits a monitor, it wakes up people from the external queue. But how do you wake up people from the internal queue?

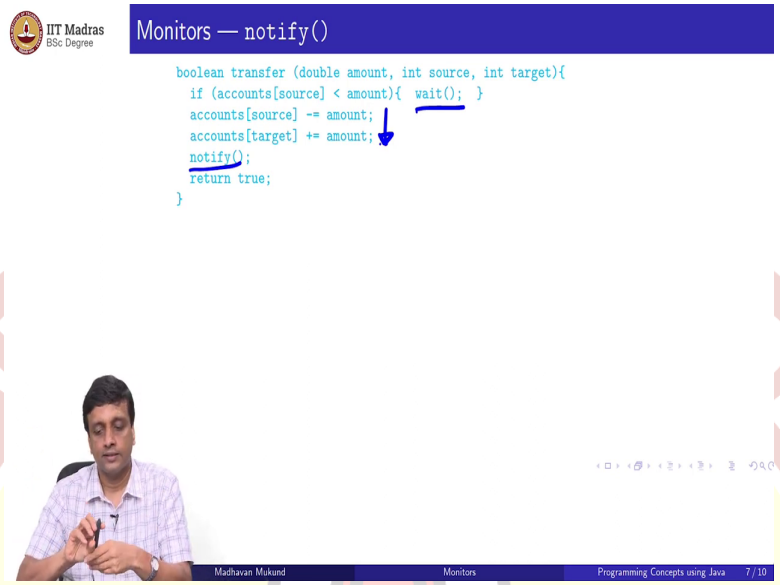
So there are two queues, remember, so exiting your function is a way of saying that nobody is occupying this monitor anymore. So, anybody who is waiting to get in for the first time is now able to get in. And then there is some priority protocol by which that thing is chosen. We do not care exactly for the moment how it happens. And we cannot predict also how it happens.

But here we are saying something different. We are saying that the state of the monitor exchange. So those people who are waiting inside can now be told that you should also be in contention. So, next time, when I wake up, I could either wake up somebody from the internal queue or the external queue. But if I have not changed the state of the monitor, then I should not do this because it is so useless to wake them up.

If you have not done anything to the bank account, supposing you are the audit trail, if you have not done anything your bank account, other than adding up the balances, it does not help anybody who is waiting for a transfer because no money has gone in or out. So, then it is useless to tell the internal queues, but to wait to tell the external. So, you need a sensible

notify operation to decide when to tell the guys who are waiting the internal queue that it is useful to wake up and check again.

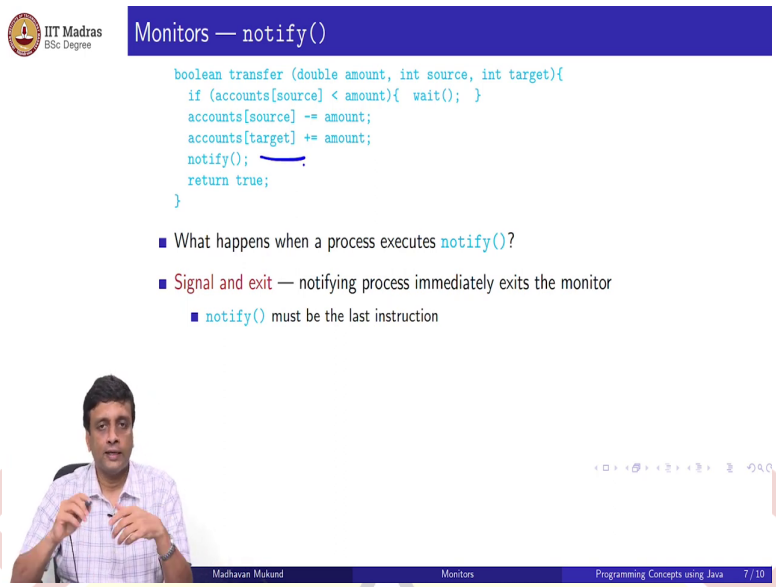
(Refer Slide Time: 13:55)



So, if I look at the code as such, before explaining what it is supposed to be doing, I need now two extra kind of primitive operations called wait and notify. So, wait is a thread suspending itself and notify is a thread telling other threads. So, after a thread succeeds in transferring, you will notify all the threads who are waiting because they are waiting for a transfer to happen.

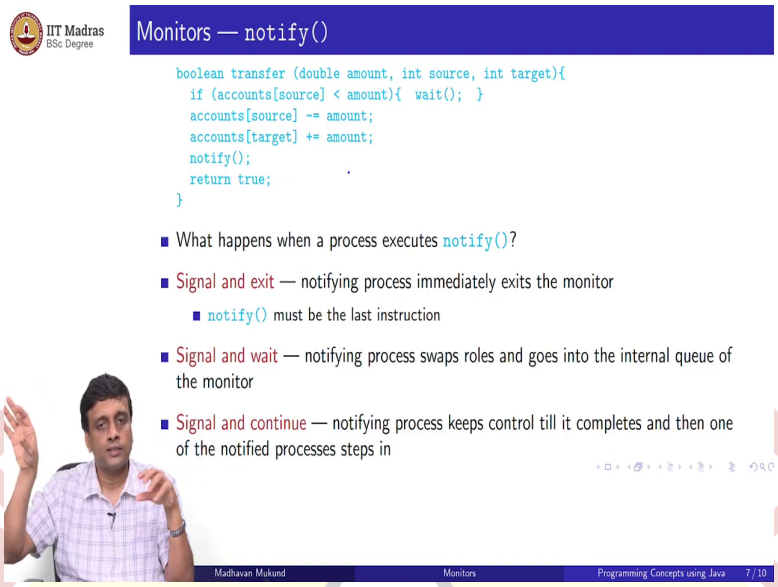
On the other hand, the audit thing presumably will not notify you not wait also because there is nothing that you get stuck when you are just adding up the accounts. And all will notify because there is no change to the state. So, the thread transfer thing does await because it is waiting for something to happen, gets out of the picture and it gets notified when some other thread succeeds. So, what does it mean to notify, well, different some monitors are a generic concept.

(Refer Slide Time: 14:45)



So, when we execute notify, there are two things or two or three things you could do one is that you immediately wake up somebody and you go that will make sense if notifies the very last thing that you do. Otherwise, you will actually gain reach a situation where I have to suspend the process which was happily progressing just because I notified somebody. So, the signal an exit is not very commonly used.

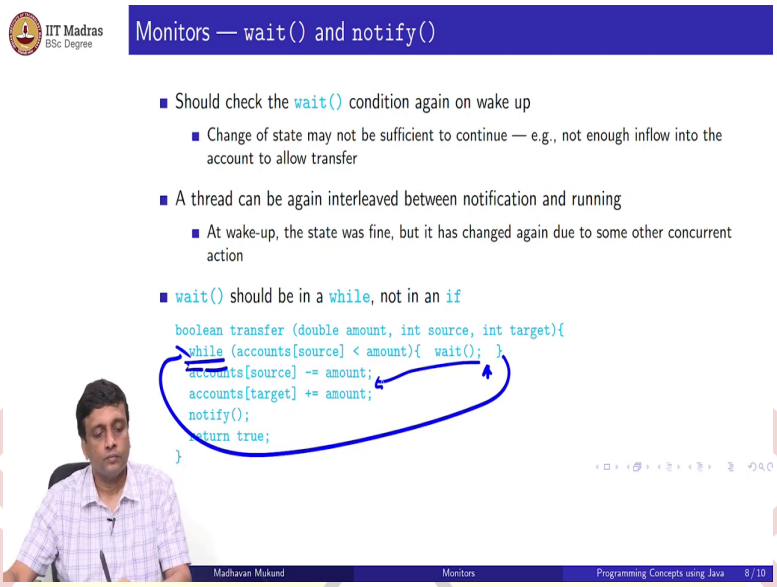
(Refer Slide Time: 15:08)

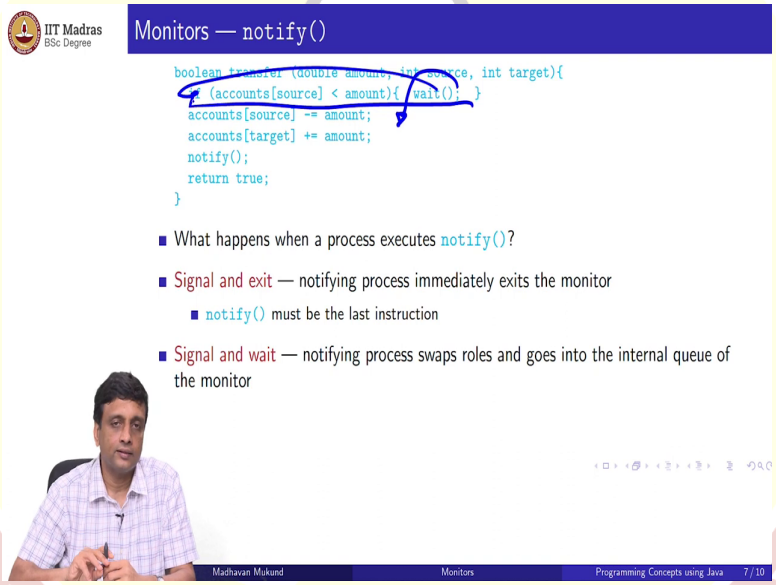


Signal and wait basically says that we swap, you are an internal queue, I let you go ahead, but then I go and wait an internal queue and then you have to let me go. And again, this requires some kind of cooperation between the two. And the last one, which is the one that is most commonly used. And we will see that Java has a version of monitors and it uses it is called signal and continue, which is that you just send out a flag saying, , all you guys were waiting, something good has happened.

So, when I am done, you should check. So, this really makes it like you have two queues, you have the external queue, which has a red light, and you have an internal queue, which is red light. Now, when this function normally finishes the external queue, red light turns green, if you do a notify, the internal queue also turns green. So, depending on which lights are green, the processes in both these queues are only in the external queue will get a chance to execute later.

(Refer Slide Time: 15:59)





So now you are sitting in this internal queue, and you are told that a transfer is happening. So, is it guarantee that you can go ahead? Well, it is not clear, supposing you wanted to transfer 400, but there was another transfer, which put only 200 into this account, then your problem is not resolved here. So, you cannot automatically assume that because you woke up, the condition that you were waiting for, has actually become true, it might be something which is not affecting you at all, it could be a transfer to another account, not the account you are waiting for or it could be insufficient.

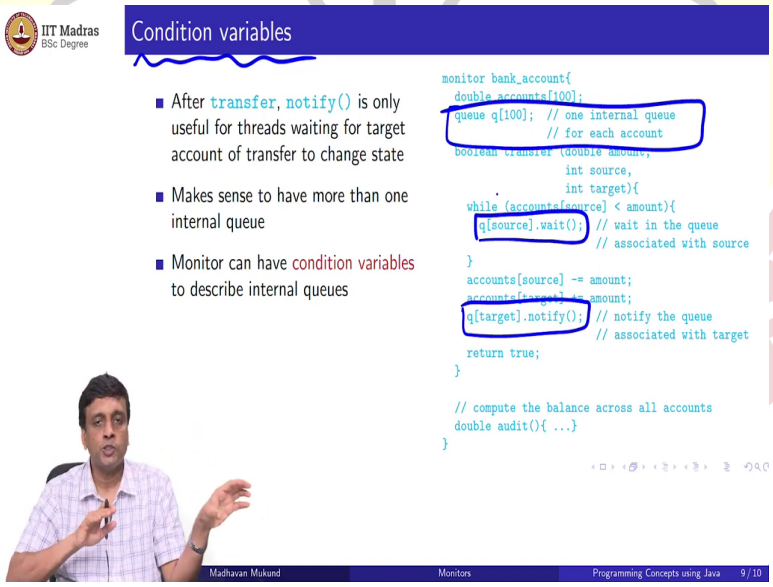
The other situation that can happen is that you wake up believing it to be true. And then by the time you do something, again, something has happened. So, you could be again,

interleaved between notification and running. So, in general, what you not need to do is make sure that you check this condition. So earlier, if you remember, we had put an if, we said if this condition is true, then wait. But what is going to happen is that when I finish waiting, I am going to come out and go ahead. So, I should not go ahead, I should go back and check this condition again, that is important.

So, the correct thing to do is to actually put it into a while. So, we have a while instead of if. So, whenever you wait for something good to happen, because your concurrent thread is stuck because of the current state of the object. In a monitor, what you should do is make sure that that condition that you check, you check again.

And the way to check it again is to make sure that when you wake up from your waiting, at this point, you come back here, other than go to the next statement. So, so that is a while is just a repetitive, you keep checking the if and as long as the if is false, you stay there. And if it becomes I mean, as long as the if is true, you stay there. So, in this case, as long as this condition that you are waiting for is not become true, you will keep waiting, and then you will proceed.

(Refer Slide Time: 17:53)



So, the other thing that we eluded to the fact that this notify could be useless simply because you have transferred into some account and you notified somebody was way. So, you have transferred into account number 17. And the actual person waiting was waiting on account number 23. So, your notify is guaranteed to not have any impact on that person's waiting.

So, you ideally, having this one internal queue is a little bit crude, because they are all waiting for different things. And if anything happens, which may or may not help any of them, all of them get woken up. So, a more focused way of doing this is to indicate what you are waiting for. So, in this particular case, we could have more than one internal queue and this internal queues and sometimes tied to the bank account for which we are expecting a change.

So we are trying to transfer 400 from j to something. So, we are only interested in changes to j you changes happen to account other than j we are not interested. So, we are in qj. Similarly, somebody has been qj prime or qk or ql. So, here, what we do is we have a separate mean, remember, this is all not real Java code, but just some pseudo code to indicate the principle. So, in our monitor, we would separately indicate an array of queues, for example.

And now, instead of just a simple wait, if I am trying to transfer money from a source account to a target account, the problem is that the source account does not have enough funds. So, I need money to come into that source account. So, that is this place where I wait. I say I am in the queue, waiting for a change to this account and the account I am waiting for is the source account.

So, the so this account is waiting for the source account. And, on the other hand, when I finish, whom do I notify? So, the person who might have to notify presumably in this context, at least is the person who is waiting for funds to come in. So, I have just transferred money from some source to some target so the person who is likely to benefit from this as the person who is waiting on the target account, so I wait on the source, but I notify the target.

So earlier, we just had a single weight and a single notify. So, everything went into one queue. So, all the processes that were all the threads that were waiting for whatever they were waiting for, will get notified. Now I have a more focused way to notify, I am saying I am waiting for something to happen on this account.

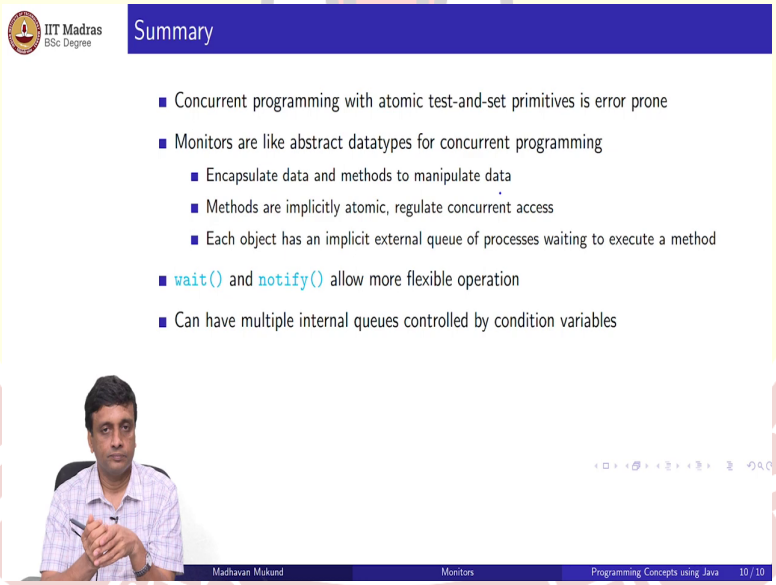
And when I finish some transfer, I say I will notify people are waiting on that target account. So, if the target account has nobody waiting, it has no effect. Otherwise, it will only wake up those people who are waiting for that target account, all the other people are waiting for other accounts to change will not get unnecessarily woken up, because nothing has changed for them. So, these are called condition variables.

So, this so for when out when I am suspending myself, I am suspending myself for a reason. Now, this is one particular method. So, you know why everybody is, everybody is suspended for the same reason, namely, that there are insufficient funds for a transfer. But that within that reason, there is sub reason which account did not have sufficient funds for transfer.

And so you can separate it according to that. In a more sophisticated example, you might have many different ways in which people get suspended abroad, threads get suspended, and each of them might be notified by different actions which rectify that problem. So, it is good to separate out the reason for the waiting. And this is what is called a condition variable.

So, when you wait, you say I am waiting for this condition to be true. And when you notify, you notify whichever conditions could have changed because of what you have done. So, then there is a match between those who are waiting, and those who make changes which update.

(Refer Slide Time: 21:35)



So, monitors are actually what Java uses. As we will see, in order to provide synchronous, remember, our whole goal is to provide programming language support for synchronization and concurrent threads. So, we started with this kind of mutual exclusion protocols, which were manufactured just from normal variables like Peterson's protocol, then we move to these test and set things, typically the semaphore things. And we said that semaphores are good, better than mutual exclusion protocols like Peterson, but still a bit low level and a little prone to error.

So then, we move to this more object oriented approach. And these are these monitors which were invented by Per Brinch Hansen and Tony Hoare. So, these are like abstract data types. The only thing is that all the methods which are stored inside a monitor, have an additional guarantee that they are mutually exclusive. So, it is a programming language promise, if you put into monitor these things will never execute in parallel. But so there is an external queue, obviously, which then has to store all the people who are waiting for access. And whenever thread finishes, it gets released and the queue gets processed and so on.

But then we saw that sometimes there may be something which is temporarily not available, because there is a batch of operations which needs to be performed, and the state may later become favorable. So, we added this flexibility of wait and notify. So, we said that a thread which is not making progress, because of some problem with the state of the object may choose to not fail, but rather to retry.

And, to retry, unfortunately, because of the semantics of a monitor. You cannot retry while waiting there because you are blocking the monitor from anybody else changing and you cannot change the state except through this monitor. So, you have to move a site to an internal queue. And then when you finish doing something, then you notify the people in the internal queue. And then we can make this final by having multiple internal queues.

So, they have this condition variables. So, you can say I am waiting for this to happen. And then when you proceed, and you make some progress, then you notify everybody whose conditioned could have changed because of you. So, you have a kind of conditional wait and a conditional notify.